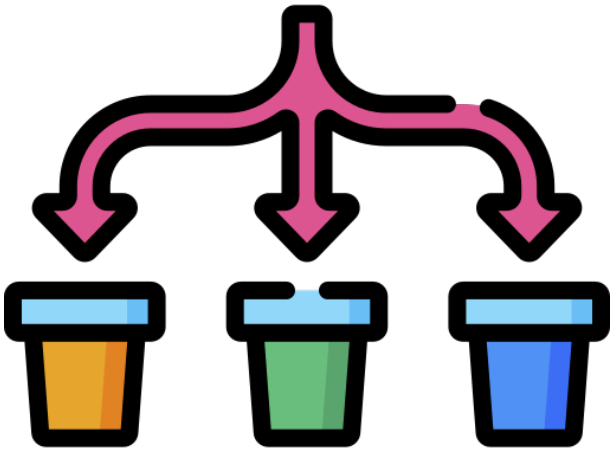
Programation orientée-objets
Prof. Dr P.-A. Mudry
Semestre de printemps 2025-2026

Informatique et systèmes de communication ISC



Benchmark de tri – SelectionSort vs YSort

Labo 16 – Tris et complexité

Comparaison entre SelectionSort et YSort

Gabriel Zeizer & Aurélien Santi

24 avril 2026

Table des matières

1 – Introduction 3
1.1 – Méthodologie 3
2 – Tests 3
3 – Résultats 3
3.1 – Tableaux aléatoires 3
3.2 – Tableaux presque triés 4
3.3 – Tableaux mélangés 4
3.4 – Tableaux inversés 5
4 – Analyse 5
4.1 – Résultats attendus vs observés 5
4.2 – Complexité observée 5
4.3 – Insensibilité au type de tableau 6
4.4 – Ratio de performance 6
5 – Conclusion 6
6 – Code Scala 7

1 – Introduction

Ce rapport présente une comparaison des performances de deux algorithmes de tri : **SelectionSort** et **YSort**. Les mesures ont été effectuées sur quatre types de tableaux : aléatoires, presque triés, mélangés et inversés, pour des tailles allant de 100 à 100 000 éléments.

1.1 – Méthodologie

Les tests ont été réalisés à l'aide de quatre factories :

- **RandomArrayFactory** : tableaux générés aléatoirement
- **AlmostSortedArrayFactory** : tableaux quasi-triés
- **ShuffleArrayFactory** : tableaux avec permutations aléatoires
- **InvertedSortedArrayFactory** : tableaux inversés

Pour chaque combinaison `(algorithme, factory, taille)`, le temps d'exécution est mesuré en millisecondes. Les résultats bruts sont disponibles dans le fichier `results.csv`.

2 – Tests

Les factories et l'algorithme SelectionSort ont été validés à l'aide des tests unitaires **ScalaTest** (`ArrayFactorySpec.scala` et `SortSpec.scala`) fournis avec le labo.

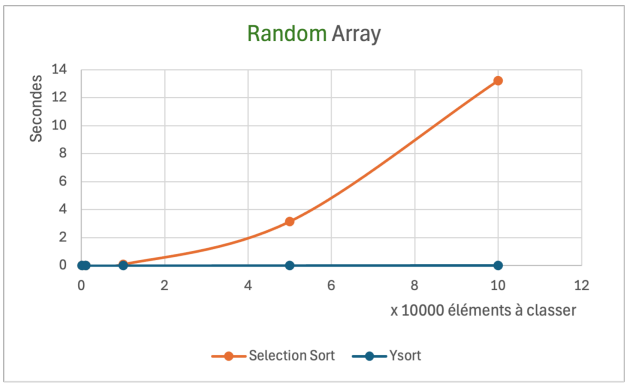
Aucune erreur n'a été relevée.

3 – Résultats

3.1 – Tableaux aléatoires

Taille	SelectionSort (ms)	YSort (ms)
100	1	2
1 000	5	1
10 000	215	4
50 000	2910	4
100 000	11829	10

Table 1 - Temps d'exécution sur `RandomArrayFactory`



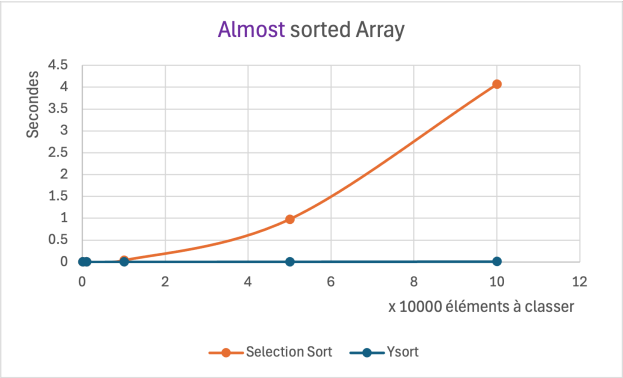
On constate une grosse différence à partir de $n = 10k$.

À $n = 100k$, SelectionSort est environ **1200x plus lent** que YSort.

3.2 – Tableaux presque triés

Taille	SelectionSort (ms)	YSort (ms)
100	0	0
1 000	0	0
10 000	29	1
50 000	908	3
100 000	3834	6

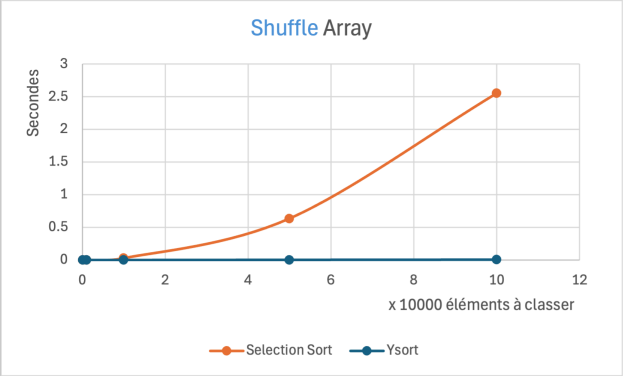
Table 2 - Temps d'exécution sur `AlmostSortedArrayFactory`



3.3 – Tableaux mélangés

Taille	SelectionSort (ms)	YSort (ms)
100	0	0
1 000	0	0
10 000	26	0
50 000	628	2
100 000	2519	4

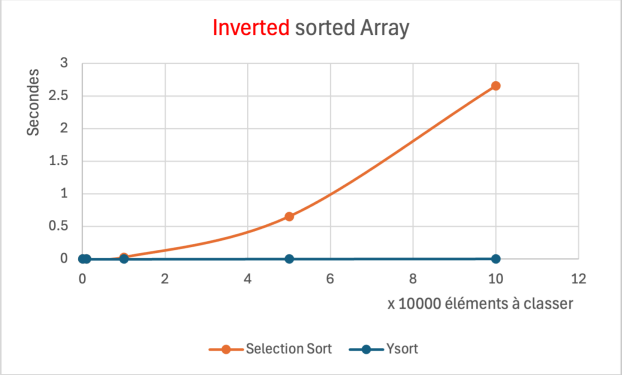
Table 3 - Temps d'exécution sur `ShuffleArrayFactory`



3.4 – Tableaux inversés

Taille	SelectionSort (ms)	YSort (ms)
100	0	0
1 000	0	0
10 000	26	1
50 000	635	2
100 000	2620	4

Table 4 - Temps d'exécution sur `InvertedSortedArrayFactory`



4 – Analyse

4.1 – Résultats attendus vs observés

On s'attendait à ce que SelectionSort soit plus lent en général, mais on observe également qu'il est aussi lent sur les tableaux presque triés que sur les aléatoires. Normalement on pourrait penser qu'un tableau déjà quasi-trié serait plus rapide à trier, mais SelectionSort parcourt toujours tout le tableau pour trouver le minimum donc ça change rien. YSort lui profite un peu des structures partiellement ordonnées, ce qui explique ses temps encore plus bas sur AlmostSorted.

4.2 – Complexité observée

SelectionSort effectue systématiquement $\frac{n(n-1)}{2}$ comparaisons, ce qui correspond à une complexité :

$$T(n) = O(n^2)$$

Cela explique la croissance quadratique observée : passer de $n = 10000$ à $n = 100000$ multiplie le temps par environ **50**, cohérent avec $10^2 = 100$ (facteur théorique).

YSort, lui, affiche une croissance quasi-linéaire sur tous les types de tableaux, compatible avec une complexité :

$$T(n) = O(n \log n)$$

4.2.1 – Temps prévu pour 10^{12} éléments

YSort a une complexité $O(n \log n)$. En se basant sur notre mesure à $n = 100k$ (10^5) ≈ 10 ms pour des tableaux aléatoires, on peut estimer $t(n)$:

$$t(n) \approx t_{10^5} \cdot \frac{n \log n}{10^5 \log 10^5}$$

Avec $n = 10^{12}$:

$$t(10^{12}) = 10[ms] \cdot \frac{10^{12} \cdot \log(10^{12})}{10^5 \cdot \log(10^5)} = 10 \cdot \frac{12 \cdot 10^7}{5} = 24 \cdot 10^7[ms] = 2.\overline{7} \text{ jours}$$

4.2.2 – SelectionSort à 10^{12} éléments

Avec $O(n^2)$, en extrapolant depuis $n = 100k$ (10^5) ≈ 11′829 ms :

$$t(n) \approx t_{10^5} \cdot \frac{(10^n)^2}{(10^5)^2}$$

Avec $n = 10^{12}$:

$$t(10^{12}) = 11829[ms] \cdot \frac{(10^{12})^2}{(10^5)^2} = 11829 \cdot \frac{10^{24}}{10^{10}} = 11829 \cdot 10^{14}[ms] \approx 375'095 \text{ siècles}$$

Autrement dit : totalement impossible et irréaliste.

4.2.3 – Un algorithme de tri peut-il être meilleur que $O(n)$?

Non. Pour trier n éléments, il faut au minimum lire chaque élément une fois. Il est impossible de trier sans connaître les valeurs contenues.

4.3 – Insensibilité au type de tableau

SelectionSort reste lent même sur des tableaux presque triés, car son mécanisme de sélection du minimum ne tire aucun avantage de l'ordre existant. YSort, en revanche, bénéficie potentiellement des structures partiellement ordonnées.

4.4 – Ratio de performance

Le rapport $\frac{T_{\text{SelectionSort}(n)}}{T_{\text{YSort}(n)}}$ sur RandomArrayFactory :

Taille	Ratio
1 000	5×
10 000	54×
50 000	728x
100 000	1183×

Table 5 - Ratio SelectionSort / YSort sur tableaux aléatoires

5 – Conclusion

YSort surpasse largement **SelectionSort** sur tous les types et tailles de tableaux testés. L'écart croît avec n, confirmant la différence de classe de complexité entre $O(n^2)$ et $O(n \log n)$. Pour tout usage pratique au-delà de quelques milliers d'éléments, SelectionSort devient inutilisable.

6 – Code Scala

```scala
import scala.util.Random

trait ArrayFactory {
  def create(size : Int) : Array[Int]
}

object InsertionSort {
  def sort(a: Array[Int]): Array[Int] = {
    for(i <- 1 until a.length){
      var j = i
      val tmp = a(i)
      while ((j > 0) && (a(j-1) > tmp)){
        a(j) = a(j-1)
        j += -1
      }
      a(j) = tmp
    }
    a
  }
}

object FisherYates {
  def mix(a: Array[Int]): Array[Int] = {
    for(i <- (a.length-1) to 1 by -1){
      val j = (new Random()).nextInt(i)
      val tmp = a(j)
      a(j) = a(i)
      a(i) = tmp
    }
    a
  }

}

object RandomArrayFactory extends ArrayFactory {
  override def create(size: Int): Array[Int] = {
    val a : Array[Int] = new Array[Int](size)
    for(i <- a.indices){
      val random = (new Random()).nextInt(size)
      a(i) = random
    }
    a
  }
}

object InvertedSortedArrayFactory extends ArrayFactory {
  override def create(size: Int): Array[Int] = {
    val a : Array[Int] = new Array[Int](size)
    for(i <- a.indices){
      a(i) = size - (i+1)
    }
    a
  }
}

object ShuffleArrayFactory extends ArrayFactory {
  override def create(size: Int): Array[Int] = {
    val a : Array[Int] = new Array[Int](size)
    var cnt = 1
    for(i <- a.indices){
      if((i % 2) == 0){
        a(i) = i/2
      } else if (i == 0) {
        a(i) = i
      } else {
        a(i) = size - cnt
        cnt += 1
      }
```

```scala
    }
    a
  }
}

object AlmostSortedArrayFactory extends ArrayFactory {
  override def create(size: Int): Array[Int] = {
    val p : Double = 0.3 // 30%
    val times : Int = (math.round(size * p)).toInt
    val a : Array[Int] = new Array[Int](size)

    for(i <- a.indices){
      a(i) = i
    }

    for(i <- 0 until times){
      var k : Int = 0
      while (k == 0){
        k = (new Random()).nextInt(a.length-1)
      }
      val j = (new Random()).nextInt(k)
      val tmp = a(j)
      a(j) = a(i)
      a(i) = tmp
    }
    a
  }
}
```

Listing 1 - Code du fichier `Task1.scala`

```scala
class SortApplication {
  def displayArray(a: Array[Int]) : Unit = {
    println("Output Array : \n" + a.mkString(" ") + "\n")
  }
}

object RunSortApplication extends App {
  val SortApp1 = new SortApplication
  println("-------------------------------------------------------")
  println("Testing ArrayFactories :")
  println("-------------------------------------------------------")
  println("RandomArrayFactory :")
  SortApp1.displayArray(RandomArrayFactory.create(20))
  println("ShuffleArrayFactory :")
  SortApp1.displayArray(ShuffleArrayFactory.create(20))
  println("AlmostSortedArrayFactory :")
  SortApp1.displayArray(AlmostSortedArrayFactory.create(20))
  println("-------------------------------------------------------")
  println("Testing Sorting Algorithm :")
  println("-------------------------------------------------------")
  println("SelectionSort - RandomArrayFactory :")
  SortApp1.displayArray(SelectionSort.sort(RandomArrayFactory.create(20)))
  println("SelectionSort - ShuffleArrayFactory :")
  SortApp1.displayArray(SelectionSort.sort(ShuffleArrayFactory.create(20)))
  println("SelectionSort - AlmostSortedArrayFactory :")
  SortApp1.displayArray(SelectionSort.sort(AlmostSortedArrayFactory.create(20)))
}
```

Listing 2 - Code du fichier `SortApplication.scala`

```scala
trait Sort {
  def sort(a: Array[Int]): Array[Int]
}

object SelectionSort extends Sort {
  override def sort(a: Array[Int]): Array[Int] = {
    // println("Input Array :\n" + a.mkString(" ") + "\n")
    for(j <- a.indices) {
      var minIndex : Int = j
      for (i <- j until a.length-1) {
        if (a(i) > a(i + 1)){
          if (a(i+1) < a(minIndex)){
            minIndex = i + 1
          }
        }
      }
      val tmp: Int = a(j)
      a(j) = a(minIndex)
      a(minIndex) = tmp
    }
    a
  }
}
```

Listing 3 - Code du fichier `Part2.scala`

```scala
import scala.collection.mutable.ArrayBuffer
import java.io._
import javax.sound.sampled.AudioSystem

class SortingTester {
  val algos: Array[Sort] = Array[Sort](SelectionSort, YSort)

  val arrayfactories: Array[ArrayFactory] =
    Array[ArrayFactory](RandomArrayFactory, AlmostSortedArrayFactory, ShuffleArrayFactory, InvertedSortedArrayFactory)

  val s: ArrayBuffer[Int] = ArrayBuffer[Int]()
  s.addOne(100)
  s.addOne(1000)
  s.addOne(10000)
  s.addOne(50000)
  s.addOne(100000)
  val size: Array[Int] = s.toArray

  def run(): Unit = {
    val musicfile2 = new File("./src/main/scala/windows-xp-startup.wav")
    val clip2 = AudioSystem.getClip()
    val audio2 = AudioSystem.getAudioInputStream(musicfile2)
    clip2.open(audio2)
    clip2.start()

    val pw = new PrintWriter(new FileOutputStream("./TestResults/results.csv"))
    pw.println("Algorithme,ArrayFactory,Taille,Temps[ms]")


    //for(reps <- 0 to 0){
      for (factory <- arrayfactories) {
        for (i <- size) {
          val a: Array[Int] = factory.create(i)
          for (algo <- algos) {
            for(reps <- 0 to 9){
              val a1: Array[Int] = a.clone()
              var algoName = algo.getClass.getSimpleName()
              var factoryName = factory.getClass.getSimpleName()
              algoName = algoName.substring(0, (algoName.length - 1))
              factoryName = factoryName.substring(0, (factoryName.length - 1))

              pw.print(algoName + ",")
              pw.print(factoryName + ",")
```

```
            pw.print(i + ",")

            println(s"Algo : $algoName - Factory : $factoryName - Size : $i")
            val st = System.currentTimeMillis()

            algo.sort(a1)

            val duration = System.currentTimeMillis() - st

            pw.println(duration)
            println("Duration : " + duration + " [ms]\n")

          }
        }
      }
    }
  //}

  val musicfile = new File("./src/main/scala/windows-xp-shutdown.wav")
  val clip = AudioSystem.getClip()
  val audio = AudioSystem.getAudioInputStream(musicfile)
  clip.open(audio)
  clip.start()
  pw.close()

  val time: Long = System.currentTimeMillis() + 7500

  //to not kill the fun
  while (System.currentTimeMillis() < time) {
    for (i <- 0 to 9) {
      Thread.sleep(15)
    }
  }
 }
}
object Tester extends App {
 val sortingTest = new SortingTester

 sortingTest.run()
}
```

Listing 4 - Code du fichier `SortingTester.scala`

```
class SizeFinder {
 val algos: Array[Sort] = Array[Sort](SelectionSort, YSort)

 val arrayfactories: Array[ArrayFactory] =
  Array[ArrayFactory](RandomArrayFactory, AlmostSortedArrayFactory, ShuffleArrayFactory, InvertedSortedArrayFactory)

 def run(): Unit = {
  var duration: Long = 0
  var lo: Int = 0
  var hi: Int = 0
  var mid: Int = 0

  for (algo <- algos) {
   val algoname: String = algo.getClass.getSimpleName
   println(s"===== ${algoname.substring(0, algoname.length - 1)} =====")

   // Choix des bornes
   if (algo == SelectionSort) {
    lo = 1000; hi = 100000
   } else {
    lo = 1000; hi = 10000000
   }

   // Recherche dichotomique
   while (hi - lo > 500) {
    mid = (lo + hi) / 2
    val a = arrayfactories(0).create(mid) // Utilisation de RandomArrayFactory
```

```scala

      val st = System.currentTimeMillis()
      algo.sort(a)
      duration = System.currentTimeMillis() - st

      if (duration < 900) {
        lo = mid
      } else if (duration > 1100) {
        hi = mid
      } else {
        lo = hi
      }

      if(hi-lo > 500) println(s"Size : $mid - Duration : ${duration}ms")
    }
    println("------- Size found ! -------")
    println(s"Size : ~$mid - Duration : ${duration}ms ≈ 1s \n")
  }
 }
}

object tableSizeFinder extends App {
 val sf = new SizeFinder
 sf.run()
}
```

Listing 5 - Code du fichier `Task8.scala`

PPPPS : Sorry pour la 11ème page